

Hungrify Food Ordering Database Description

The Hungrify Food Ordering Database is a compact, AI-ready relational schema designed to manage users, restaurants, menu items, orders, and deliveries while enabling intelligent personalization, allergen detection, and operational analytics. At its foundation lies the `users` table, which stores all types of system participants—customers, restaurant owners, delivery partners, and administrators. It includes essential fields such as `username`, `password`, `email`, `phone`, and `roles` (as JSON for flexible access control). Additional attributes like `verified`, `active`, and `last_login` allow tracking engagement and security. The `admin` table maintains a one-to-one relationship with users, extending it with administrative configurations, permissions, and dashboard settings for system management.

The `restaurants` table captures key business details, including the restaurant's name, cuisine type, address, city, and geographic coordinates. It also holds `restaurant_meta`, a JSON field used to store delivery zones, operational preferences, and special attributes such as "halal" or "pure-veg". Each restaurant record links to an owner in the `users` table, ensuring proper account association and ownership management.

Central to the schema is the `food_items` table, which stores each menu item's core information such as display name, descriptions, price, availability, preparation time, and images. It includes a rating column for user feedback and a quantity field for stock tracking. For intelligent search, the schema uses a FULLTEXT index on name and description, enabling the system to interpret natural-language queries like "find spicy vegan dishes under ₹300". Each food item is enriched by a corresponding record in the `food_item_profiles` table (a 1:1 mapping), which serves as the AI feature layer. This table includes nutritional data such as calories, proteins, and fats, along with fields like `spice_score`, `spice_level`, `tags`, and `allergens` (stored as JSON). It also maintains computed boolean flags (`has_milk`, `has_peanuts`, `is_vegan`, `is_gluten_free`) generated from the allergen list. These computed and indexed attributes make allergen-aware filtering and recommendation queries both simple and fast.

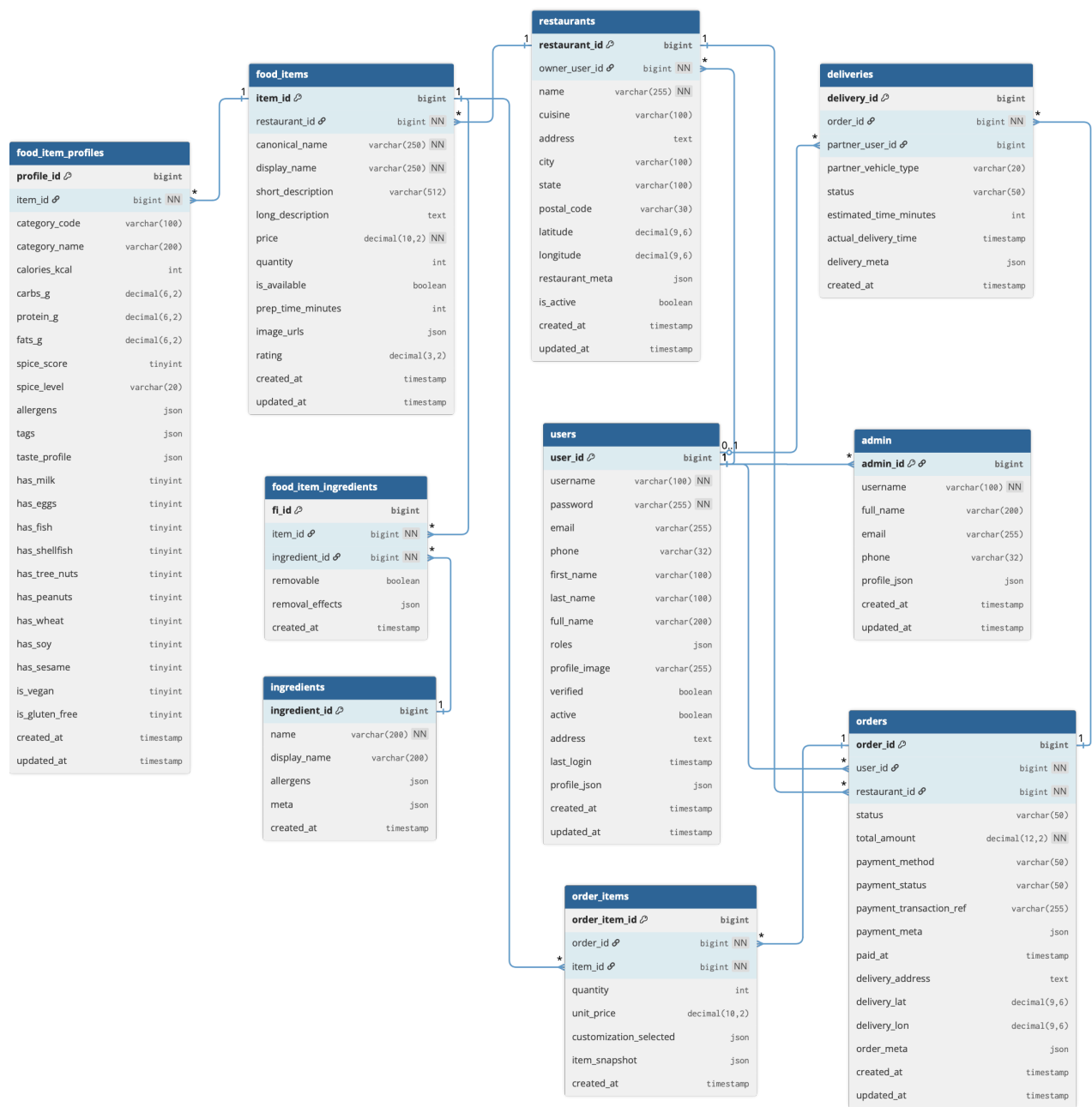
The `ingredients` table lists all raw ingredients with their corresponding allergen metadata, while the `food_item_ingredients` table acts as a junction that connects items with ingredients, specifying whether an ingredient is removable. This design allows the system to determine whether a dish can be modified to suit dietary preferences, such as making it dairy-free or nut-free, by analyzing the `removal_effects` JSON field.

The `orders` table records every customer purchase with full context, linking to the user and restaurant involved. It captures order status, total amount, and embedded payment information including `payment_method`, `payment_status`, and `transaction_ref`, removing the need for a separate payment table. The `order_items` table represents individual dishes within an order and includes the ordered quantity, unit price, customizations, and an `item_snapshot` JSON field that preserves the item's allergen and nutritional profile at the time of purchase. This ensures historical traceability and accurate analytics even if menu data changes later.

Each order is connected to a single record in the `deliveries` table, forming a one-to-one relationship. Deliveries track which delivery partner handled the order, their vehicle type, delivery status, and estimated versus actual delivery times. Additional data such as GPS coordinates or ETA predictions can be stored in `delivery_meta` for performance analysis. This structure makes it easy to monitor the delivery lifecycle—from assignment to completion—and calculate delivery efficiency or delay metrics.

Together, these interconnected tables form a seamless data flow that mirrors the complete customer experience—from discovering restaurants and browsing personalized menus to placing orders, tracking deliveries, and receiving intelligent recommendations. The schema is optimized for both transactional efficiency and analytical depth, using JSON fields for flexibility, generated attributes for allergen detection, and strategic indexing for fast AI-driven querying. By integrating user behavior, nutritional data, and operational metrics, the Hungrify database transforms raw order and menu data into actionable insights. It empowers real-time recommendations, dietary-aware filtering, and performance analytics, making the platform both highly adaptive for users and data-rich for administrators—all within a cohesive, scalable, and intelligent design.

ER- Diagram



Data Directory

Table Name	Attribute Name	Contents	Type	Format	Range	Required	PK OR FK	FK referenced
users	user_id	Unique user identifier	BIGINT	Auto-increment integer	1-9,999,999	Y	PK	
	username	Unique login username	VARCHAR(100)	Text	1-100 chars	Y		
	password	Password hash (hashed)	VARCHAR(255)	BCrypt hash	1-255 chars	Y		
	email	User email address	VARCHAR(255)	user@example.com	Valid email, <=255 chars	N		
	phone	User phone number	VARCHAR(32)	+CC-XXXXXXXX	<=32 chars	N		
	first_name	User first name	VARCHAR(100)	Text	1-100 chars	Y		
	last_name	User last name	VARCHAR(100)	Text	1-100 chars	Y		
	full_name	Concatenated first and last name	VARCHAR(200)	Text	1-200 chars	N		
	roles	User roles array	JSON	JSON object	App-defined structure	Y		
	profile_image	Profile image filename or URL	VARCHAR(255)	filename.jpg or URL	<=255 chars	N		
	verified	Verified flag	BOOLEAN	Boolean	0/1	N		
	active	Active flag	BOOLEAN	Boolean	0/1	N		
	address	User address	TEXT	Free text	Free text	N		
	last_login	Last login timestamp	TIMESTAMP	YYYY-MM-DD HH:MM:SS	Valid date/time or NULL	N		
	profile_json	User preferences and personalization	JSON	JSON object	App-defined structure	N		
	created_at	Record creation timestamp	TIMESTAMP	YYYY-MM-DD HH:MM:SS	Auto-set timestamp	N		
	updated_at	Record last update timestamp	TIMESTAMP	YYYY-MM-DD HH:MM:SS	Auto-set timestamp	N		
admin	admin_id	Admin user identifier (1:1 to users)	BIGINT	Integer	1-9,999,999	Y	PK	users.user_id
	username	Admin username	VARCHAR(100)	Text	1-100 chars	Y		
	full_name	Admin full name	VARCHAR(200)	Text	1-200 chars	N		
	email	Admin email address	VARCHAR(255)	user@example.com	Valid email, <=255 chars	Y		
	phone	Admin phone number	VARCHAR(32)	+CC-XXXXXXXX	<=32 chars	N		
	profile_json	Admin role and dashboard config	JSON	JSON object	App-defined structure	N		
	created_at	Record creation timestamp	TIMESTAMP	YYYY-MM-DD HH:MM:SS	Auto-set timestamp	N		
	updated_at	Record last update timestamp	TIMESTAMP	YYYY-MM-DD HH:MM:SS	Auto-set timestamp	N		
restaurants	restaurant_id	Unique restaurant identifier	BIGINT	Auto-increment integer	1-9,999,999	Y	PK	
	owner_user_id	Owner user id	BIGINT	Integer	1-9,999,999	Y	FK	users.user_id
	name	Restaurant name	VARCHAR(255)	Text	1-255 chars	Y		
	cuisine	Cuisine type	VARCHAR(100)	Text	1-100 chars	Y		
	address	Restaurant address	TEXT	Free text	Free text	N		
	city	City name	VARCHAR(100)	Text	1-100 chars	Y		
	state	State or region	VARCHAR(100)	Text	1-100 chars	N		
	postal_code	Postal or ZIP code	VARCHAR(30)	Text	1-30 chars	N		
	latitude	Latitude coordinate	DECIMAL(9,6)	Decimal degrees	-90.000000 to 90.000000	Y		
	longitude	Longitude coordinate	DECIMAL(9,6)	Decimal degrees	-180.000000 to 180.000000	Y		
	restaurant_meta	Restaurant metadata	JSON	JSON object	App-defined structure	N		
	is_active	Active flag	BOOLEAN	Boolean	0/1	N		
	created_at	Record creation timestamp	TIMESTAMP	YYYY-MM-DD HH:MM:SS	Auto-set timestamp	N		
	updated_at	Record last update timestamp	TIMESTAMP	YYYY-MM-DD HH:MM:SS	Auto-set timestamp	N		
ingredients	ingredient_id	Unique ingredient identifier	BIGINT	Auto-increment integer	1-9,999,999	Y	PK	
	name	Ingredient canonical name	VARCHAR(200)	Text	1-200 chars	Y		
	display_name	Ingredient display name	VARCHAR(200)	Text	1-200 chars	N		
	allergens	List of allergens	JSON	JSON object	App-defined structure	N		
	meta	Ingredient metadata	JSON	JSON object	App-defined structure	N		
	created_at	Record creation timestamp	TIMESTAMP	YYYY-MM-DD HH:MM:SS	Auto-set timestamp	N		
food_items	item_id	Unique food item identifier	BIGINT	Auto-increment integer	1-9,999,999	Y	PK	
	restaurant_id	Owning restaurant id	BIGINT	Integer	1-9,999,999	Y	FK	restaurants.restaurant_id
	canonical_name	Canonical internal name	VARCHAR(250)	Text	1-250 chars	Y		
	display_name	Display name for menu	VARCHAR(250)	Text	1-250 chars	Y		
	short_description	Short item description	VARCHAR(512)	Text	1-512 chars	N		
	long_description	Detailed item description	TEXT	Free text	Free text	N		
	price	Item price	DECIMAL(10,2)	Decimal (10,2)	Monetary amount	Y		
	quantity	Default quantity	INT	Integer	-2147483648 to 2147483647	N		
	is_available	Availability flag	BOOLEAN	Boolean	0/1	N		
	prep_time_minutes	Preparation time in minutes	INT	Integer	-2147483648 to 2147483647	N		
	image_urls	List of image URLs	JSON	JSON object	App-defined structure	N		
	rating	Average rating	DECIMAL(3,2)	Decimal (3,2)	0.00-9.99	N		
	created_at	Record creation timestamp	TIMESTAMP	YYYY-MM-DD HH:MM:SS	Auto-set timestamp	N		
	updated_at	Record last update timestamp	TIMESTAMP	YYYY-MM-DD HH:MM:SS	Auto-set timestamp	N		

Table Name	Attribute Name	Contents	Type	Format	Range	Required	PK OR FK	FK referenced
food_item_profiles	profile_id	Unique profile identifier	BIGINT	Auto-increment integer	1-9,999,999	Y	PK	
	item_id	Related food item id (1:1)	BIGINT	Integer	1-9,999,999	Y	FK	food_items. item_id
	category_code	Category code	VARCHAR(100)	Text	1-100 chars	N		
	category_name	Category display name	VARCHAR(200)	Text	1-200 chars	N		
	calories_kcal	Calories in kcal	INT	Integer	0-2147483647	N		
	carbs_g	Carbohydrates in grams	DECIMAL(6,2)	Decimal (6,2)	>=0	N		
	protein_g	Protein in grams	DECIMAL(6,2)	Decimal (6,2)	>=0	N		
	fats_g	Fats in grams	DECIMAL(6,2)	Decimal (6,2)	>=0	N		
	spice_score	Spice score 0-5	TINYINT	Integer	0-5	N		
	spice_level	Spice level enum	ENUM('none', 'mild','medium', 'spicy','very_spicy')	One of: [none,mild,medium,spicy, very_spicy]	One of enumerated values	N		
	allergens	List of allergens	JSON	JSON object	App-defined structure	N		
	tags	Tags array	JSON	JSON ARRAY object	App-defined structure	N		
	taste_profile	Taste profile object	JSON	JSON object	App-defined structure	N		
	has_milk	Derived flag: contains milk	BOOLEAN	Boolean	0/1	N		
	has_eggs	Derived flag: contains eggs	BOOLEAN	Boolean	0/1	N		
	has_fish	Derived flag: contains fish	BOOLEAN	Boolean	0/1	N		
	has_shellfish	Derived flag: contains shellfish	BOOLEAN	Boolean	0/1	N		
	has_tree_nuts	Derived flag: contains tree nuts	BOOLEAN	Boolean	0/1	N		
	has_peanuts	Derived flag: contains peanuts	BOOLEAN	Boolean	0/1	N		
	has_wheat	Derived flag: contains wheat	BOOLEAN	Boolean	0/1	N		
	has_soy	Derived flag: contains soy	BOOLEAN	Boolean	0/1	N		
	has_sesame	Derived flag: contains sesame	BOOLEAN	Boolean	0/1	N		
	is_vegan	Derived vegan flag	BOOLEAN	Boolean	0/1	N		
	is_gluten_free	Derived gluten-free flag	BOOLEAN	Boolean	0/1	N		
	created_at	Profile creation timestamp	TIMESTAMP	YYYY-MM-DD HH:MM:SS	Auto-set timestamp	N		
	updated_at	Profile last update timestamp	TIMESTAMP	YYYY-MM-DD HH:MM:SS	Auto-set timestamp	N		
food_item_ingredients	fi_id	Junction record identifier	BIGINT	Auto-increment integer	1-9,999,999	Y	PK	
	item_id	Related food item id	BIGINT	Integer	1-9,999,999	Y	FK	food_items. item_id
	ingredient_id	Related ingredient id	BIGINT	Integer	1-9,999,999	Y	FK	ingredients. ingredient_id
	removable	Removable flag	BOOLEAN	Boolean	0/1	N		
	removal_effects	Effects when removed	JSON	JSON object	App-defined structure	N		
	created_at	Record creation timestamp	TIMESTAMP	YYYY-MM-DD HH:MM:SS	Auto-set timestamp	N		
orders	order_id	Unique order identifier	BIGINT	Auto-increment integer	1-9,999,999	Y	PK	
	user_id	Ordering user id	BIGINT	Integer	1-9,999,999	Y	FK	users.user_id
	restaurant_id	Restaurant id for order	BIGINT	Integer	1-9,999,999	Y	FK	restaurants. restaurant_id
	status	Order status	ENUM('pending', 'confirmed', 'preparing', 'out_for_delivery', 'delivered', 'cancelled')	One of: [pending,confirmed, preparing,out_for_delivery, delivered,cancelled]	One of enumerated values	N		
	total_amount	Total payable amount	DECIMAL(12,2)	Decimal (12,2)	Monetary amount	Y		
	payment_method	Payment method	ENUM ('credit_card', 'debit_card','upi', 'wallet','cod')	One of: [credit_card,debit_card, upi,wallet,cod]	One of enumerated values	N		
	payment_status	Payment status	ENUM('pending', 'paid','failed', 'refunded')	One of: [pending,paid,failed, refunded]	One of enumerated values	N		
	payment_transaction_ref	Payment transaction reference	VARCHAR(255)	Text	<=255 chars	N		
	payment_meta	Payment metadata	JSON	JSON object	App-defined structure	N		
	paid_at	Payment timestamp	TIMESTAMP	YYYY-MM-DD HH:MM:SS	Valid date/time or NULL	N		
	delivery_address	Delivery address	TEXT	Free text	Free text	N		
	delivery_lat	Delivery latitude	DECIMAL(9,6)	Decimal degrees	-90.000000 to 90.000000	N		
	delivery_lon	Delivery longitude	DECIMAL(9,6)	Decimal degrees	-180.000000 to 180.000000	N		
	order_meta	Order metadata	JSON	JSON object	App-defined structure	N		
	created_at	Order creation timestamp	TIMESTAMP	YYYY-MM-DD HH:MM:SS	Auto-set timestamp	N		
	updated_at	Order last update timestamp	TIMESTAMP	YYYY-MM-DD HH:MM:SS	Auto-set timestamp	N		

Table Name	Attribute Name	Contents	Type	Format	Range	Required	PK OR FK	FK referenced
order_items	order_item_id	Order line item identifier	BIGINT	Auto-increment integer	1-9,999,999	Y	PK	
	order_id	Parent order id	BIGINT	Integer	1-9,999,999	Y	FK	orders.order_id
	item_id	Ordered item id	BIGINT	Integer	1-9,999,999	Y	FK	food_items.item_id
	quantity	Quantity ordered	INT	Integer	-2147483648 to 2147483647	N		
	unit_price	Unit price at order time	DECIMAL(10,2)	Decimal (10,2)	Monetary amount	N		
	customization_selected	Selected customizations	JSON	JSON object	App-defined structure	N		
	item_snapshot	Snapshot of item details	JSON	JSON object	App-defined structure	Y		
deliveries	created_at	Line item creation timestamp	TIMESTAMP	YYYY-MM-DD HH:MM:SS	Auto-set timestamp	N		
	delivery_id	Unique delivery identifier	BIGINT	Auto-increment integer	1-9,999,999	Y	PK	
	order_id	Associated order id (1:1)	BIGINT	Integer	1-9,999,999	Y	FK	orders.order_id
	partner_user_id	Delivery partner user id	BIGINT	Integer	1-9,999,999	N	FK	users.user_id
	partner_vehicle_type	Partner vehicle type	ENUM('bike','car','bicycle','other')	One of: [bike,car,bicycle,other]	One of enumerated values	N		
	status	Delivery status	ENUM('assigned','picked_up','delivered','cancelled')	One of: [assigned,picked_up,delivered,cancelled]	One of enumerated values	N		
	estimated_time_minutes	Estimated delivery minutes	INT	Integer	-2147483648 to 2147483647	N		
	actual_delivery_time	Actual delivery timestamp	TIMESTAMP	YYYY-MM-DD HH:MM:SS	Valid date/time or NULL	N		
	delivery_meta	Delivery metadata	JSON	JSON object	App-defined structure	N		
	created_at	Record creation timestamp	TIMESTAMP	YYYY-MM-DD HH:MM:SS	Auto-set timestamp	N		

BIGINT	Large integer numeric data type used for IDs or counters
VARCHAR	Variable-length character string (1–2,000 characters)
CHAR	Fixed-length character string (1–255 characters)
TEXT	Large block of text or unstructured string data
JSON	JavaScript Object Notation structure for storing arrays, preferences, or objects
TINYINT	Boolean or small integer value (commonly used for flags like active/verified)
TIMESTAMP	Date and time value including timezone, used for logs or record tracking
NUMBER(9,2)	Numeric data type with precision 9 and scale 2 (e.g., currency, sales amount)
PK	Primary key (unique record identifier for a table)
FK	Foreign key (links to a primary key in another table)

Only key columns displayed for sample data visualization										
Table Name	Users									
Primary Key	user_id									
Foreign Key	None									
user_id	username	email	phone	first_name	last_name	verified	active	last_login	created_at	
1	alice.w	alice.w@example	911000000001	Alice	Walsh	1	1	2025-10-20 9:12:00	2025-01-10 8:00:00	
2	bob.k	bob.k@example	911000000002	Bob	Kumar	1	1	2025-10-22 18:00:00	2025-02-14 10:30:00	
3	charlie.s	charlie.s@examp	911000000003	Charlie	Shah	0	1	NULL	2025-03-03 12:00:00	
4	diana.r	diana.r@example	911000000004	Diana	Rao	1	1	2025-10-21 20:15:00	2025-04-01 11:00:00	
5	eric.m	eric.m@example	911000000005	Eric	Menon	1	1	2025-09-30 7:45:00	2025-05-02 8:00:00	
6	frank.o	frank.o@example	911000000006	Frank	Obero	1	1	2025-10-10 13:00:00	2025-06-05 9:00:00	
7	gina.t	gina.t@example	911000000007	Gina	Thomas	1	1	2025-10-23 7:00:00	2025-07-12 10:00:00	
Table Name	admin									
Primary Key	admin_id									
Foreign Key	admin_id → users(user_id)									
admin_id	username	full_name	email	phone	profile_json	created_at	updated_at			
1	alice.admin	Alice Walsh	alice.admin@exa	911000000001	{permissions*: ["view_reports"]}	2025-01-10 8:05	2025-10-20 9:20:00			
4	diana.admin	Diana Rao	diana.admin@ex	911000000004	{permissions*: ["manage_menus"]}	2025-04-01 11:0	2025-10-21 20:16:00			
Table Name	restaurants									
Primary Key	restaurant_id									
Foreign Key	owner_user_id → users(user_id)									
restaurant_id	owner_user_id	name	cuisine	city	state	postal_code	latitude	longitude	is_active	
1	4	Diana's Deli	Italian	San Francisco	California	94103	37.78635	-122.404	1	
2	6	Frank's Burgers	American	Los Angeles	California	90026	34.0814	-118.2607	1	
3	9	Irene's Spice Co	Indian	San Francisco	California	94112	37.7213	-122.4381	1	
4	1	Alice's Greens	Vegetarian	Los Angeles	California	90015	34.0438	-118.265	1	
5	6	Frank's Late Nigh	American	San Diego	California	92109	32.8029	-117.2417	1	
6	11	Kiran's Kitchen	South Indian	San Diego	California	92126	32.9154	-117.1386	1	
Table Name	ingredients									
Primary Key	ingredient_id									
Foreign Key	NONE									
ingredient_id	name	display_name	allergens	meta	created_at					
1	tomato	Tomato	[]	{type*: "vegetab	2025-01-05 9:00					
2	mozzarella	Mozzarella Chee	[milk]*	{origin*: "buffalo	2025-01-06 9:00					
3	basil	Fresh Basil	[]	{use*: "garnish"}	2025-01-07 9:00					
4	wheat_flour	Wheat Flour	[wheat]*	{gluten*: true}*	2025-01-08 9:00					
5	chicken	Chicken	[]	{type*: "poultry"}	2025-01-09 9:00					
6	garlic	Garlic	[]	{}	2025-01-10 9:00					
7	peanuts	Peanuts	[peanuts]*	{roasted*: true}*	2025-01-11 9:00					
Table Name	food_items									
Primary Key	item_id									
Foreign Key	restaurant_id → restaurants(restaurant_id)									
item_id	restaurant_id	canonical_name	display_name	price	quantity	is_available	prep_time_minu	rating	created_at	
1	1	margherita_pizza	Margherita Pizza	59	1	1	18	4.5	2025-04-06 10:00:00	
2	1	pasta_arrabbiata	Pasta Arrabbiata	34	1	1	12	4.2	2025-04-06 10:05:00	
3	2	classic_cheeseb	Classic Cheeseb	39	1	1	15	4.3	2025-06-06 11:00:00	
4	2	fries_large	Large Fries	19	1	1	6	4	2025-06-06 11:05:00	
5	3	butter_chicken	Butter Chicken	49	1	1	25	4.6	2025-08-21 12:00:00	
6	3	spicy_shrimp	Spicy Shrimp Cu	549	1	1	20	4.25	2025-08-21 12:10:00	
7	4	quinoa_salad	Quinoa Salad	29	1	1	10	4.1	2025-01-12 9:40:00	
Table Name	food_item_profiles									
Primary Key	profile_id									
Foreign Key	item_id → food_items(item_id)									

Primary Key	order_id									
Foreign Key	user_id → users(user_id)									
	restaurant_id → restaurants(restaurant_id)									
order_id	user_id	restaurant_id	status	total_amount	payment_method	payment_status	paid_at	delivery_address	created_at	
1001	1	1	delivered	898	upi	paid	2025-10-18 12:30	742 Market Street, San Francisco, CA	2025-10-18 12:00:00	
1002	2	2	out_for_delivery	528	wallet	paid	2025-10-22 18:10	1242 Sunset Boulevard, Los Angeles, CA	2025-10-22 17:50:00	
1003	3	3	preparing	648	credit_card	pending	NULL	5195 Mission Street, San Francisco, CA	2025-10-23 11:00:00	
1004	5	4	confirmed	478	upi	paid	2025-10-19 13:00	1200 Grand Avenue, Los Angeles, CA	2025-10-19 12:45:00	
1005	6	2	delivered	728	debit_card	paid	2025-10-10 23:30	801 N Alvarado Street, Los Angeles, CA	2025-10-10 23:00:00	
1006	7	5	cancelled	598	cod	pending	NULL	1024 Garnet Avenue, San Diego, CA	2025-10-23 7:10:00	
1007	8	6	preparing	268	upi	paid	2025-10-22 8:45	8930 Mira Mesa Blvd, San Diego, CA	2025-10-22 8:30:00	
Table Name	order_items									
Primary Key	order_item_id									
Foreign Key	order_id → orders(order_id)									
	item_id → food_items(item_id)									
order_item_id	order_id	item_id	quantity	unit_price	customization	item_snapshot	created_at			
5001	1001	1	1	599	{(extra": "basil")}	{(price": 599.00	"item_id": 1	"display_name": "Margherita Pizza")"	2025-10-18 12:01:00	
5002	1001	13	1	199	{}	{(price": 199.00	"item_id": 13	"display_name": "Tomato Bruschetta")"	2025-10-18 12:02:00	
5003	1002	3	1	399	{(doneness": "medium")}	{(price": 399.00	"item_id": 3	"display_name": "Classic Cheeseburger")"	2025-10-22 17:52:00	
5004	1002	4	1	129	{}	{(price": 129.00	"item_id": 4	"display_name": "Large Fries")"	2025-10-22 17:53:00	
5005	1003	5	1	499	{(spice": "medium")}	{(price": 499.00	"item_id": 5	"display_name": "Butter Chicken")"	2025-10-23 11:01:00	
5006	1003	14	1	49	{}	{(price": 49.00	"item_id": 14	"display_name": "Butter Naan")"	2025-10-23 11:02:00	
5007	1004	7	1	299	{(dressing": "lemon")}	{(price": 299.00	"item_id": 7	"display_name": "Quinoa Salad")"	2025-10-19 12:46:00	
Table Name	deliveries									
Primary Key	delivery_id									
Foreign Key	order_id → orders(order_id)									
	partner_user_id → users(user_id)									
delivery_id	order_id	partner_user_id	partner_vehicle	status	estimated_time	actual_delivery	delivery_meta	created_at		
9001	1001	2	bike	delivered	25	2025-10-18 13:00	{(note": "contacted")}	2025-10-18 12:05:00		
9002	1002	7	bike	picked up	15	NULL	{(eta": "15_min")}	2025-10-22 18:00:00		
9003	1003	NULL	NULL	assigned	20	NULL	{}	2025-10-23 11:02:00		
9004	1004	2	car	assigned	30	NULL	{}	2025-10-19 12:46:00		
9005	1005	7	bike	delivered	28	2025-10-10 23:50	{(tip": 50)}	2025-10-10 23:01:00		
9006	1006	NULL	NULL	assigned	35	NULL	{}	2025-10-23 7:12:00		
9007	1007	2	bike	assigned	18	NULL	{}	2025-10-22 8:32:00		